ATML Report

Pietro Bernardo Maria Forni *pietro.forni@usi.ch*
Luigi Tisci *tiscil@usi.ch*
Matteo Vitali *vitalm@usi.ch*

Source code: github.com/FlowReg-ATMLproject-USI

Abstract

This project studies *Flowing Through States: Neural ODE Regularization for Reinforcement Learning*, which proposes FlowReg, a training-time regularizer that aligns the latent trajectory of a reinforcement-learning agent with the trajectory generated by a Neural ODE. Because the original implementation was not available, we re-implemented the method on top of Stable-Baselines3, using PPO on MiniGrid as the main controlled reproduction and A2C on Atari as an exploratory extension. In the final MiniGrid summaries, FlowReg improves aggregate training reward on FourRooms enviroment. Atari results are more fragile: the paper-faithful regularizer is sensitive in high-dimensional visual control, so we decide to treat Atari mainly as a partial reproduction plus stabilized ablation.

1 Introduction

Modern deep reinforcement-learning agents usually act on a learned latent representation rather than on compact symbolic states. This makes representation learning part of the control problem itself: if the latent space is poorly aligned with the environment dynamics, the policy may still obtain reward locally while relying on a geometry that is hard to model, unstable, or not semantically meaningful. The paper *Flowing Through States: Neural ODE Regularization for Reinforcement Learning* addresses this issue by introducing FlowReg, an auxiliary regularizer that encourages a sequence of latent states to follow the trajectory of a Neural ODE.

Our project is a reproducibility study of this idea. Since the paper's code was not released, we built the training pipeline from scratch. The strongest controlled evidence in our results comes from PPO experiments on the MiniGrid task used in the paper. We also explored an Atari A2C implementation, and we will discuss in detail the reasons why we think that, in the Atari setting, it is harder to reproduce the paper's results faithfully.

2 Scope of reproducibility

The original paper studies latent misalignment in sequential decision-making: environment states evolve according to the MDP dynamics, but the latent embeddings learned by the agent are usually unconstrained. FlowReg addresses this by learning a Neural ODE in latent space and penalizing the discrepancy between the encoder trajectory and the ODE trajectory. In the repository, the main reproducibility target is the paper's PPO-on-MiniGrid setting, while Atari/A2C is treated as a secondary, partial extension. The claims we test are:

- **Claim 1:** A FlowReg objective of the form $L_{\text{task}} + \lambda L_{\text{flow}}$ can regularize latent trajectories toward a smooth Neural ODE path while remaining a training-only mechanism, so inference uses the same policy architecture as the baseline.
- **Claim 2:** In the paper's MiniGrid setting, PPO + FlowReg improves training performance over baseline PPO when using index time sampling, update frequency U-20, and $\lambda = 1$.

- **Claim 3:** The same latent-dynamics regularization idea transfers to visual Atari control, where A2C + FlowReg should outperform or at least remain competitive with baseline A2C.

Section 4 can then separate the evidence for each claim: Claim 1 is tested directly by the implementation and latent diagnostics, Claim 2 by the MiniGrid summaries, and Claim 3 only partially by the exploratory Atari runs and ablations contained in our code.

3 Methodology

As aforementioned, we implemented the method from scratch. The implementation is based on `stable-baselines3==2.8.0`, `gymnasium==1.2.3`, `minigrid==3.1.0`, `torch==2.12.0`, and `torchdiffeq==0.2.5`, as listed in `pyproject.toml`. The main code lives in `src/flowreg/`. Training entry points are provided for baseline PPO, FlowReg PPO, baseline A2C, and FlowReg A2C, and post-hoc analysis is handled by `flowreg-analyze-runs`.

3.1 Mathematical formulation

The method is defined on a Markov Decision Process $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma)$. Given a state sequence $\mathbf{s} = (s_0, \ldots, s_{N-1})$, the agent encoder maps each state to a latent vector through $h_\theta : \mathcal{S} \to \mathbb{R}^d$, producing the discrete latent trajectory

$$H_\theta(\mathbf{s}) = \big(h_\theta(s_0), h_\theta(s_1), \ldots, h_\theta(s_{N-1})\big).$$

FlowReg introduces a Neural ODE

$$\frac{dz(t)}{dt} = f_\phi(z(t), t),$$

initialized at the first latent state, $z(\tau_0) = h_\theta(s_0)$, and integrated on an artificial time grid $\{\tau_i\}_{i=0}^{N-1}$. In the main MiniGrid setting used in this repository, the paper-default index sampling is used, so $\tau_i = i$. This produces a continuous latent path

$$H_\phi(\mathbf{s}) = \mathrm{ODESolve}\big(f_\phi, h_\theta(s_0), \{\tau_i\}_{i=0}^{N-1}\big).$$

The regularizer penalizes the discrepancy between the latent path produced by the policy and the one generated by the ODE:

$$L_{\text{flow}}(\mathbf{s}) = \frac{1}{N} \left\| H_\theta(\mathbf{s}) - H_\phi(\mathbf{s}) \right\|_2^2 .$$

Training then combines the reinforcement-learning objective with this alignment term,

$$L = L_{\text{task}} + \lambda L_{\text{flow}},$$

so the encoder and the ODE are optimized jointly. Importantly, the ODE is used only during training: at evaluation time, the policy acts exactly like the corresponding baseline agent.

3.2 Model descriptions

MiniGrid PPO baseline. The main controlled reproduction uses Stable-Baselines3 PPO with `MlpPolicy`. MiniGrid observations are converted to image-only observations with `ImgObsWrapper` and then flattened, so the FourRooms policy receives a 147-dimensional input. The learned feature extractor is a two-layer MLP, $147 \to 128 \to 64$, with `tanh` activations. Instantiating the current FourRooms configuration yields 27,200 feature-extractor parameters and 44,360 policy parameters in total. The model is trained from scratch and end-to-end.

MiniGrid PPO + FlowReg. The FlowReg MiniGrid agent keeps the same PPO policy and adds a separate Neural ODE model used only during training. In the current main configs, the regularizer is applied to the shared extracted features (`representation: features`), so the ODE operates on a 64-dimensional latent state. The ODE network is a small MLP, $64 \to 64 \to 64$, with a `tanh` hidden layer, for 8,320 parameters. The loss compares a length-5 latent trajectory to the ODE-solved path and adds λL_{flow} to the PPO objective. The ODE is saved separately as `flow_model.pt` and is not used during evaluation.

Atari A2C baseline. The repository also contains an Atari baseline based on Stable-Baselines3 A2C with `CnnPolicy`. The observation tensor has shape $84 \times 84 \times 4$ after preprocessing and frame stacking, and the policy uses the standard Nature CNN to produce 512-dimensional features. For the current Breakout configuration, the feature extractor contains 1,684,128 parameters and the full policy contains 1,686,693 parameters. This model is also trained from scratch.

Atari A2C + FlowReg (paper-faithful target). The paper-faithful Atari target keeps the same A2C backbone and adds FlowReg as a training-time regularizer on top of the 512-dimensional Nature CNN features. In this formulation, the ODE is autonomous, i.e. it follows $dz/dt = f_\phi(z)$ without receiving the action as an input, and the loss follows the trajectory-level discrepancy described in the paper. This is the intended reproduction setting, together with the paper hyperparameters: RMSProp, initial learning rate 7×10^{-4} with linear decay, gradient clipping 0.5, and ODE tolerances `rtol=1e-4` and `atol=1e-5`.

Atari A2C + FlowReg (stabilized exploratory variant). Because the strict Atari reproduction was fragile in the experiments, the implemented exploratory variant makes the regularizer easier to optimize. It uses an action-conditioned ODE $dz/dt = f_\phi(z, a)$, a 512-dimensional latent state, hidden size 256, `mse_mean` loss reduction, delayed activation after 50,000 timesteps, and a smaller `lambda_flow=0.5`.

3.3 Datasets

The data is generated online by the environments during training. Each experiment is defined by an environment, a timestep budget, a seed, and an evaluation protocol.

MiniGrid environments. The paper's MiniGrid suite is represented in the repository through `MiniGrid-FourRooms-v0`. The final multi-seed scripts use 10 seeds per agent and the default 1M timesteps from the YAML configs. Observations follow the pipeline

$$\texttt{MiniGrid env} \rightarrow \texttt{ImgObsWrapper} \rightarrow \texttt{FlattenObservation} \rightarrow \texttt{MlpPolicy},$$

which removes mission text and yields flat image vectors. Training trajectories are sampled on-policy from 4 parallel environments, and evaluation uses 10 episodes. The exact environments are obtainable through the dependencies listed in `pyproject.toml`.

Atari environments. For Atari, the repository contains configs and scripts for `ALE/Breakout-v5` and `ALE/Qbert-v5`, even though the we tried a larger Atari benchmark. The environment factory explicitly sets ALE `frameskip: 1` and `repeat_action_probability: 0.0`, then relies on the SB3 Atari wrapper with `frame_skip: 4`, and finally applies `VecFrameStack(n_stack=4)`. As in MiniGrid, there is no static split; the training data are on-policy rollouts, and the config evaluates over 10 episodes with `deterministic: false`.

3.4 Hyperparameters

For MiniGrid, the base PPO hyperparameters come from the repository configs and follow conservative Stable-Baselines3 defaults: learning rate 3×10^{-4}, $n_steps = 256$, batch size 256, $n_epochs = 4$, $\gamma = 0.99$, GAE $\lambda = 0.95$, clip range 0.2, entropy coefficient 0.0, value-loss coefficient 0.5, and max gradient norm 0.5, with $n_envs = 4$. The MiniGrid feature extractor uses `features_dim=64` and `hidden_dim=128`.

The FlowReg-specific MiniGrid hyperparameters are taken from the paper whenever they are stated clearly and then encoded in the current YAML files: `lambda_flow=1.0`, sequence length 5, FlowReg batch size 32, index time sampling, update frequency U-20, `rtol=1e-4`, `atol=1e-5`, and an ODE hidden size of 64. The current main configs implement U-20 relative to PPO train calls (`update_unit: train_call`), which is closer to the paper's notion of periodic policy updates.

For Atari, the baseline A2C hyperparameters follow the paper more closely: learning rate 7×10^{-4} with linear decay, $n_steps = 5$, $\gamma = 0.99$, GAE $\lambda = 1.0$, entropy coefficient 0.01, value-loss coefficient 0.25, max gradient norm 0.5, and 16 parallel environments.

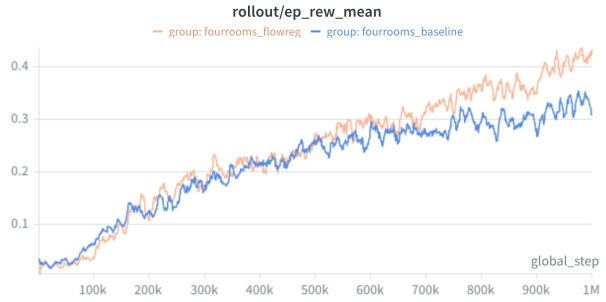


Figure 1: Mean reward per episode. Mini-Grid FourRooms Enviroment average of 10 seeds.

For the exploratory Atari setup, the FlowReg config is not strictly paper-faithful; it reduces the regularization strength to `lambda_flow=0.3`, uses `mse_mean` instead of the paper-scaled loss, delays the start of FlowReg to 50,000 timesteps, increases the ODE hidden size to 256, updates every 5 train calls, and conditions the ODE on actions. These changes are treated as stabilized ablations rather than as the final paper setting.

3.5 Experimental setup and code

The full code is contained in the accompanying repository. The main training commands described in the `README.md`. Each run stores a `config.json` snapshot, checkpoints, monitor CSV files, and TensorBoard logs under `runs/<group>/<run_id>/`. Analysis is reproduced with `flowreg-analyze-runs`, which aggregates monitor statistics and TensorBoard scalars into Markdown summaries such as `summary.md` and `new_envs_summary.md`.

The primary performance measure is episodic return. The repository reports mean reward over all completed training episodes, mean reward over the last 20 episodes, the number of positive-reward episodes, and the best observed training reward. For short local validation runs, it also uses 10-episode checkpoint evaluation with a fixed seed. FlowReg adds latent diagnostics that are central to Claim 1: `Loss/FlowReg`, `Latent/Path_Length`, `Latent/Net_Displacement`, `Latent/Acceleration_Energy`, and `Latent/ODE_Error_Drift`. The tests in `tests/test_flowreg.py` verify that sampled trajectories do not cross episode boundaries and that gradients reach both the policy and the ODE parameters.

3.6 Computational requirements

The hardware we used is a MacBook Pro with Apple M3 Pro, 12 CPU cores, 18 GPU cores, and 18 GB unified memory. MiniGrid configs use `device: cpu`, while the Atari configs default to `device: mps` for local runs. The setup notes recommend NVIDIA cloud hardware such as RTX 3090/4090, L4, or A10G with at least 16 GB VRAM for full 1M-step MiniGrid runs and any serious Atari training.

4 Results

It seems that the performance of the method depends on the environment it is tested on. While it performs well on the FourRooms Mini-Grid environment, it does not on Atari environments.

4.1 Results reproducing original paper

4.1.1 Mini-Grid

The single FourRooms enviroment, seems to prove Claim 2 from Section 2, as the performance of the regularized agent are better, as it can be seen from 1, 2.

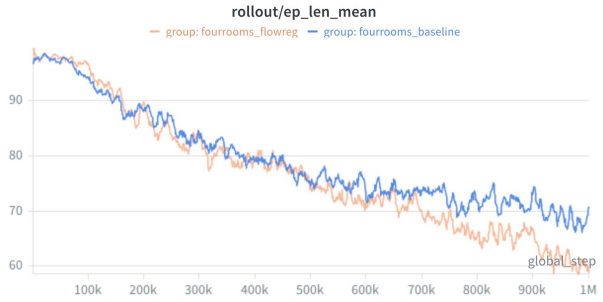


Figure 2: Mean length of episode. Mini-Grid FourRooms Enviroment average of 10 seeds.

4.2 Paper-Faithful Atari

The Atari reproduction required additional experimentation because the paper does not fully specify several engineering choices that strongly affect performance in visual RL. In particular, the exact wrapper stack, the interaction between ALE-level frame skipping and the Stable-Baselines3 Atari wrapper, sticky-action handling, reward-processing details, and the precise reduction used to translate the trajectory-level squared error into code are not fully determined by the paper. To keep the comparison controlled, we used the same A2C backbone for the baseline and FlowReg agents, matched the main paper hyperparameters.

The paper-faithful FlowReg run keeps the autonomous ODE formulation, $dz/dt = f_\phi(z)$, and applies the regularizer directly to the 512-dimensional Nature CNN features using the paper-scaled trajectory loss and `lambda_flow = 1`. This experiment is intended to test Claim 3 as directly as possible: whether the original Atari FlowReg formulation improves over the A2C baseline under a modern SB3/Gymnasium/ALE stack. The evidence collected so far by our experiments suggests that this configuration is highly sensitive: the baseline still learns under the controlled preprocessing, while the strict FlowReg setting is often weaker and can degrade training. In Figure 3, we therefore report the rollout curves of baseline A2C and paper-faithful FlowReg side by side and we see that the flowreg variant is underperfomring the baseline.

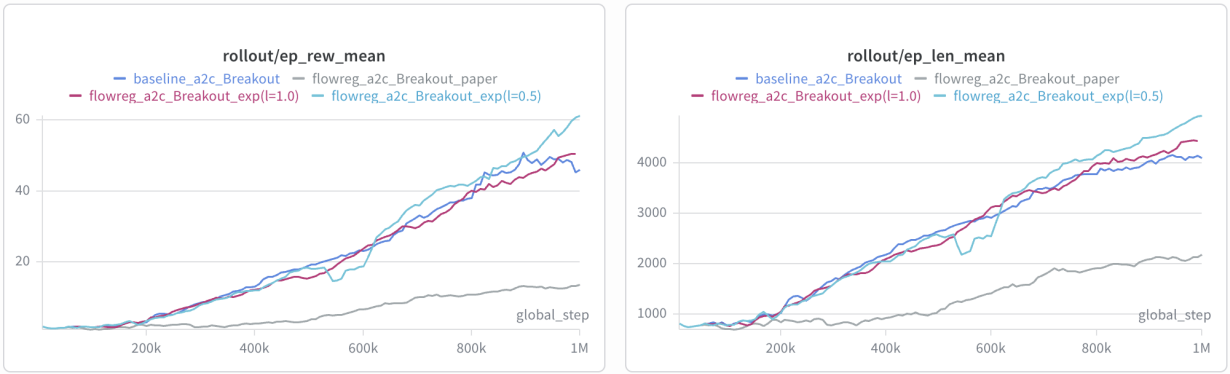


Figure 3: Rollout/ep_rew_mean comparison in Atari game BreakOut

4.2.1 Stabilized Exploratory Variant

To understand whether the Atari difficulty comes from a bug or from the optimization properties of the original objective, we tried several different configurations and each of them tends to improve reducing the λ_{flow} factor never outperforming the baseline, this make us realize that the problem was that the flowreg was just inserting noise in the equation, in particular for more complex games. So, we evaluate a stabilized exploratory variant. This new model keeps the same A2C backbone but modifies the regularizer: it replaces

the paper-scaled loss with `mse_mean`, reduces the regularization strength to `lambda_flow = 0.5`, delays the start of FlowReg to `50 000` timesteps, increases the ODE hidden size to 256, and conditions the ODE on the selected action. The action-conditioned formulation is especially important conceptually, because Atari transitions are not determined by the latent state alone; different actions from visually similar states can lead to very different next observations.

This variant is not a strict reproduction of the paper and in fact we reported it as an ablation. Its role is diagnostic: if the baseline rollout is healthy, the paper-faithful FlowReg rollout is unstable, and the stabilized variant recovers a curve closer to the baseline and tuning the λ_{flow} also superior. For this reason, the most informative visualization is a direct rollout comparison among the three agents: baseline A2C, paper-faithful FlowReg, and the stabilized exploratory variant shown in Figure 3, where we can see a qualitative proof of the idea of the paper for this more complex envirnments.

We also tested this stabilized variant on more complex Atari games, such as `Qbert`, where it remained close to the baseline instead of collapsing, which further suggests that the main difficulty is the increasing complexity and action-dependence of the environment dynamics rather than a simple implementation bug.

5 Discussion

Our results offer mixed support for the paper's claims. While we successfully verified that FlowReg functions as a training-time regularizer (Claim 1) and boosts performance in structured grid-worlds like MiniGrid `FourRooms` (Claim 2), our evidence challenges its out-of-the-box transferability to high-dimensional Atari visual control (Claim 3).

The mathematical formulation of the regularizer was clear and straightforward to implement using the `torchdiffeq` API. However, reproduction was severely hindered by the complete absence of an official code repository or any experimental guidelines. The paper omits crucial engineering choices, which made the writing of the code from scratch even more comlpex and time consuming.

Deducing these undocumented variables by trial-and-error required extensive tuning, which was heavily constrained by our limited computational power. For instance, being restricted to a single local machine, running multi-million step exploratory configurations to debug the performance degradation in `Qbert` and `Breakout` took days. Consequently, our hardware limits prevented us from scaling our multi-seed ablation campaign further to definitively map the method's stability boundaries.

6 Conclusion

In this reproducibility study, we implemented FlowReg from scratch to evaluate its effectiveness as a latent trajectory regularizer in reinforcement learning. We successfully verified its benefits in some structured grid-world environments, observing clear performance improvements in MiniGrid `FourRooms`. However, applying the autonomous ODE method to high-dimensional Atari games proved challenging, revealing a high sensitivity to undocumented hyperparameter choices and complex environment dynamics. Our reproduction was primarily limited by the total absence of the original codebase and restricted computational resources. Future extensions should involve testing our action-conditioned ODE variant across a broader suite of environments with greater compute, as well as exploring dynamic scheduling for the regularization penalty to improve stability in visual control tasks.

Member contributions

The project was a collaborative effort.